

less architecture and multicasting to bring high-quality QoS to applications. DDS can be deployed in platforms ranging from low-footprint devices to the cloud, and supports efficient bandwidth usage as well as agile orchestration of system components.

DDS protocol has two main layers: data-centric publish-subscribe (DCPS) and data-local reconstruction layer (DLRL). DCPS delivers the information to subscribers, and DLRL provides an interface to DCPS functionalities, enabling sharing of distributed data among IoT-enabled objects.

Simple Text Oriented Messaging Protocol (STOMP)

This text-based protocol was developed to work with message-oriented middleware. It provides an interoperable wire format that enables STOMP clients to communicate with any STOMP message broker to enable easy and widespread messaging interoperability among many languages, platforms and brokers. Like AMQP, STOMP provides message header with properties and a frame body.

However, STOMP does not deal in queues and topics—it uses a SEND semantic with a 'destination' string. The broker must map it onto something that it understands internally, such as a topic, queue or exchange. Consumers then subscribe to those destinations. Since those destinations are not mandated in the specifications, different brokers may support different flavours of the destination. So, it's not always straightforward to port code between brokers.

However, STOMP is simple and lightweight (although somewhat verbose on the wire), with a wide range of language bindings. It also provides

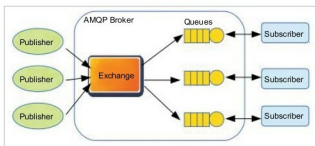


Fig. 5: AMQP architecture

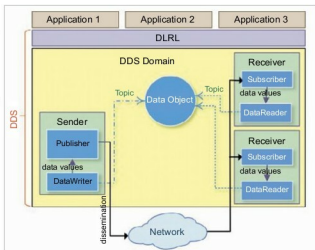


Fig. 6: DDS protocol architecture

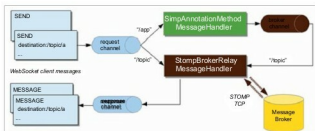


Fig. 7: STOMP architecture

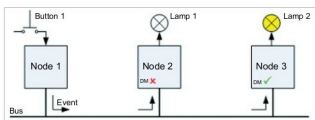


Fig. 8: VSCP operations

some transactional semantics. One of the most interesting examples is RabbitMQ Web STOMP, which allows you to expose messaging in a browser through Websockets.

Very Simple Control Protocol (VSCP). VSCP is more a framework

than a protocol. It is highly scalable, has a low footprint and is a free-cum-open source solution for device discovery and identification, device configuration, autonomous device functionality, secure firmware updates and connection of devices from sensor to smart user interface. VSCP makes things interact at the application layer. It makes use of CAN, RS-232, Ethernet, TCP/IP, MQTT and 6LoWPAN.

VSCP uses an event format and supports global unique identifiers for nodes, thus making a node identifiable no matter where it is installed in the world. Besides, it includes a register model in order to provide a flexible common interface for node configuration and a model for controlling the functionality of each node. VSCP does not make any assumptions regarding the lower level system used to realise the physical interconnection with the node. Therefore it works with different transport mechanisms such as Ethernet, TCP/IP, wireless, Zigbee, Bluetooth, CAN, GPRS, RS-232 and USB.

VSCP is event-based. Every time an event occurs, it is broadcast to all other nodes on the network. From there on, each node decides on its own if the event received needs to be processed or not. The final decision depends on the node's decision matrix, which is made up of a number of 'if condition > then action > lines', where the condition > is evaluated based on the fields present in the VSCP datagram broadcast to the network. **EFY**

This is a reprint of the article published in June issue of Open Source For You magazine.